TransLink Transit Intelligence Pipeline

INFO 4190- PROJECT INTEGRATION | PROGRESS REPORT 2

Group 9

Inderpreet Kaur 100383066

Drishti Kapoor 100435109

Table of Contents

1. Project Title and Team Info 2
2. Design Completion Status 2
3. Development Planning 10
4. Technical Challenges 11
5. Response to Peer Review #1 12
6. Next Steps 12
7. Appendix 13

1. Project Title and Team Info

Project Title: TransLink Transit Intelligence Pipeline: Data Engineering and Machine Learning System for Transit Schedule Analysis and Commuter Insights in Metro Vancouver

Name	Student ID	Role
Inderpreet Kaur	100383066	Technical Design Lead, system architecture, data pipeline design, database schema, ML model design, API design
Drishti Kapoor	100435109	Design & Documentation Lead, UI/UX wireframes, mockups, requirements specification, literature review, progress reports, final report writing

2. Design Completion Status

All core system design work is complete. The database is modeled with a full Entity Relationship diagram covering 12 tables routes, trips, stops, stop_times, calendar, calendar_dates, shapes, transfers, directions, feed_info, route_clusters, and etl_runs, with all relationships and cardinality fully defined.

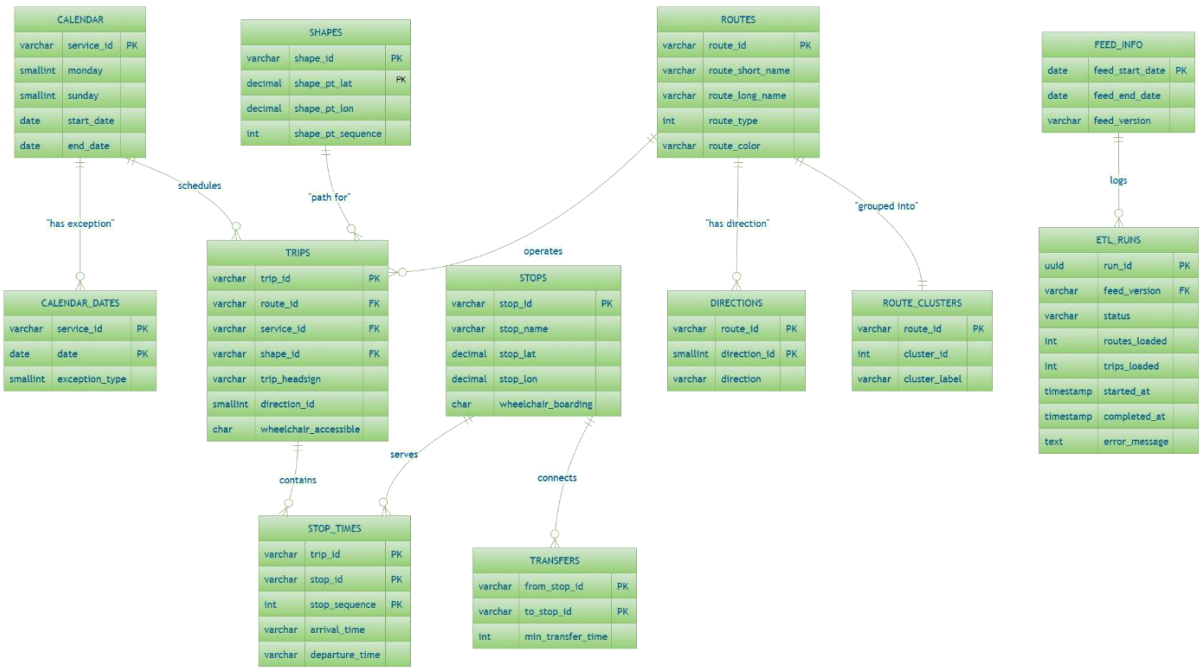


Figure 1: Database ERD

This is supported by a domain class diagram and a service-layer class diagram for the GTFS Importer component, which together define the object structure behind the pipeline. The domain class diagram defines the core system entities (routes, trips, stops, and services) and their relationships, forming the foundation of the GTFS data model.

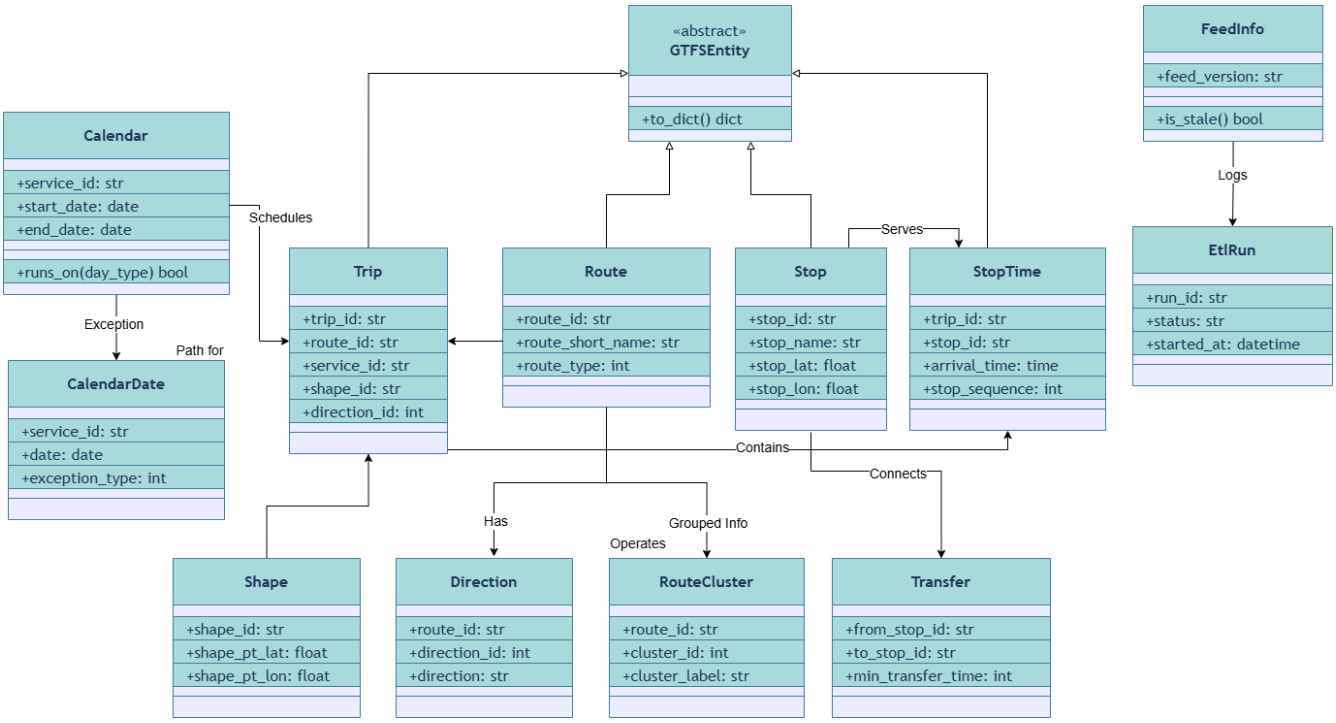


Figure 2: Domain Class Diagram

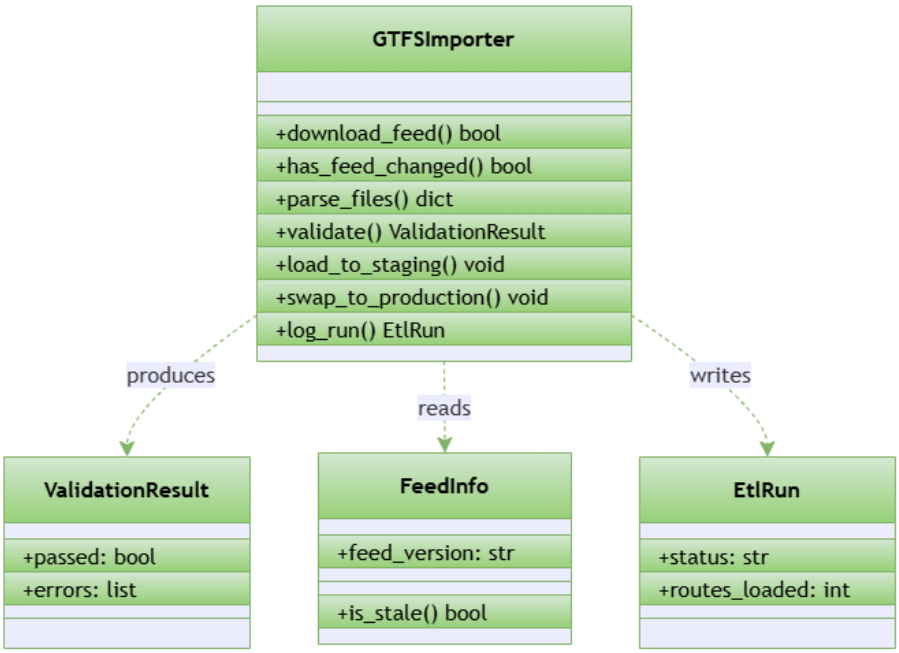


Figure 3: Service Importer Class Diagram

The four layers map directly onto the technology stack: Ingest (Python, requests library, weekly cron trigger), Process/ETL (Pandas for transformation, PostgreSQL for storage), Analyse (Scikit-Learn K-Means, optional), and Present (Flask API serving a Chart.js dashboard). Each layer is independent, so it can be built and tested on its own before the next layer depends on it.

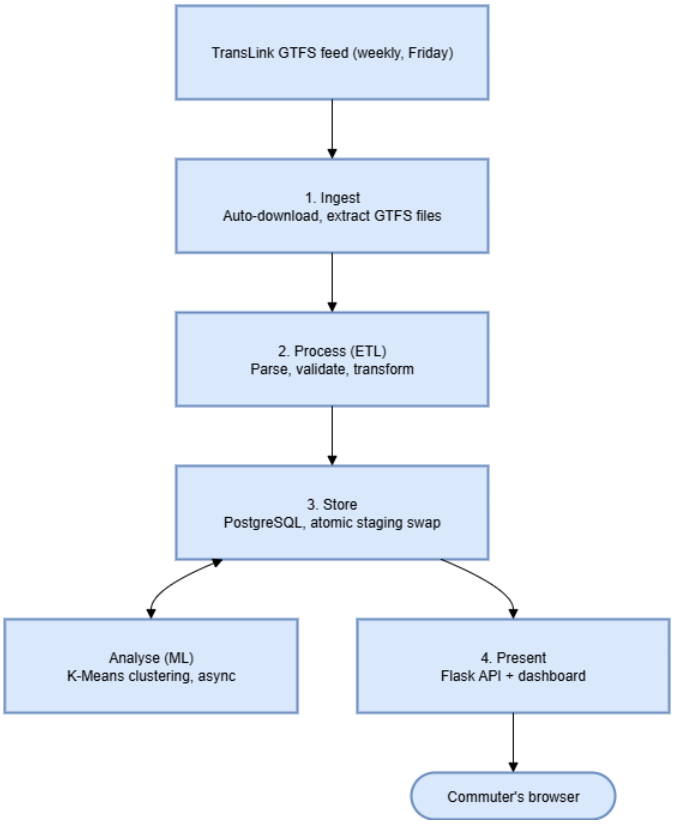


Figure 4: System Architecture Diagram

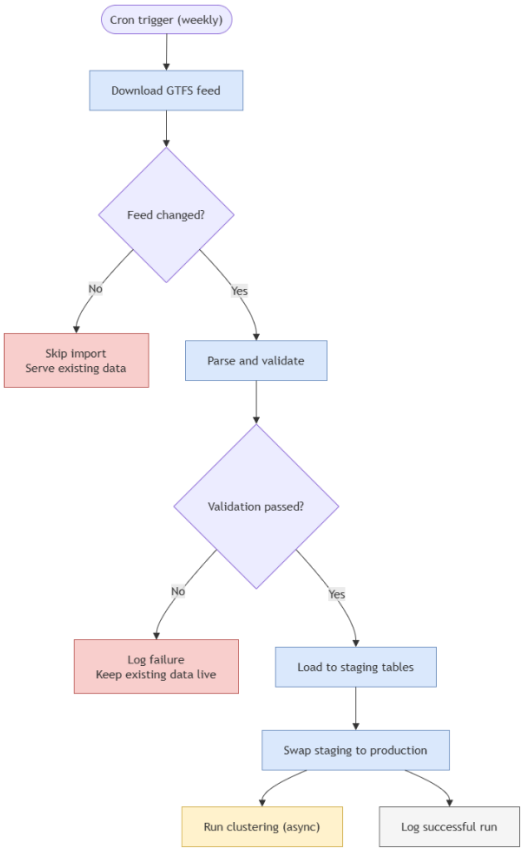


Figure 5: ETL Pipeline Flowchart

Figure 5 shows ETL Pipeline Flowchart. The pipeline runs on a weekly cron trigger. It downloads the latest GTFS feed and checks whether it has actually changed since the last run, if not, the import is skipped and the existing data keeps serving the dashboard, avoiding unnecessary reprocessing. If the feed changed, it's parsed and validated; a failed validation logs the failure and keeps the existing data live rather than corrupting production. A passing validation loads into staging tables, then atomically swaps staging into production, followed by an async clustering step and a logged successful run

Three actor groups interact with the system. Commuters can search a route or stop, view the network overview, plan a trip, and check data freshness; searching a route or stop extends into viewing frequency analysis or stop patterns, which include getting a departure suggestion, filtering results, and filtering by accessibility. The Scheduler actor runs the GTFS import, which includes validating and loading the feed and extends into running the optional clustering step. Data Enthusiasts and Researchers/Planners view the resulting network clusters.

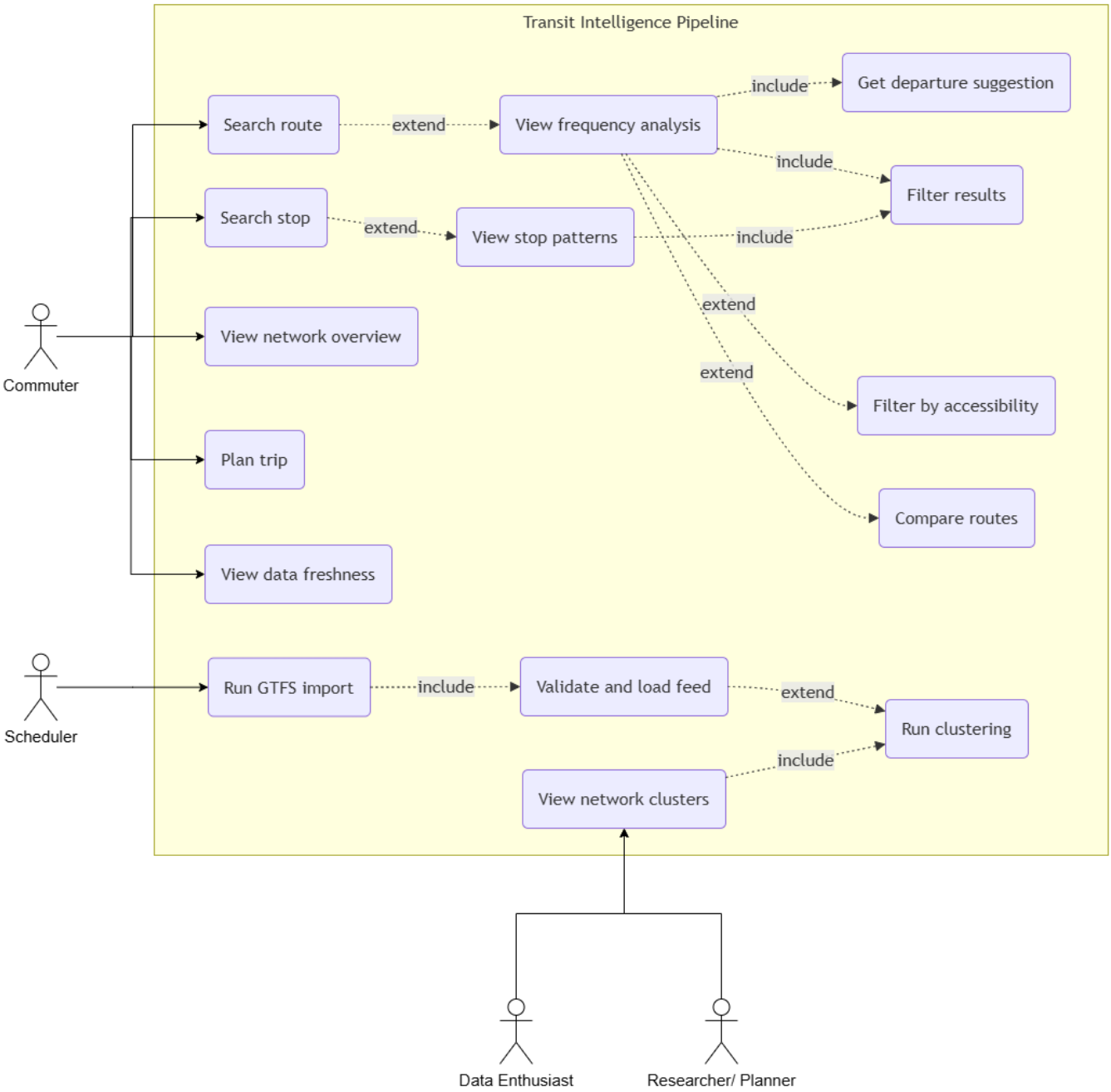


Figure 6: Use Case Diagram

Figure 7 below is a Route Analyser Sequence Diagram. A Commuter selects a route, day, and direction on the Dashboard, which requests frequency data from the API. The API calls RouteAnalyserService, which queries PostgreSQL for matching trips. The diagram splits into two paths: if the route isn't found, a 404 error is returned and shown to the user; if it is found, the service queries stop times, computes frequency/peak windows/summary, and returns a OK response, which the dashboard renders as a chart with the plain-language summary.

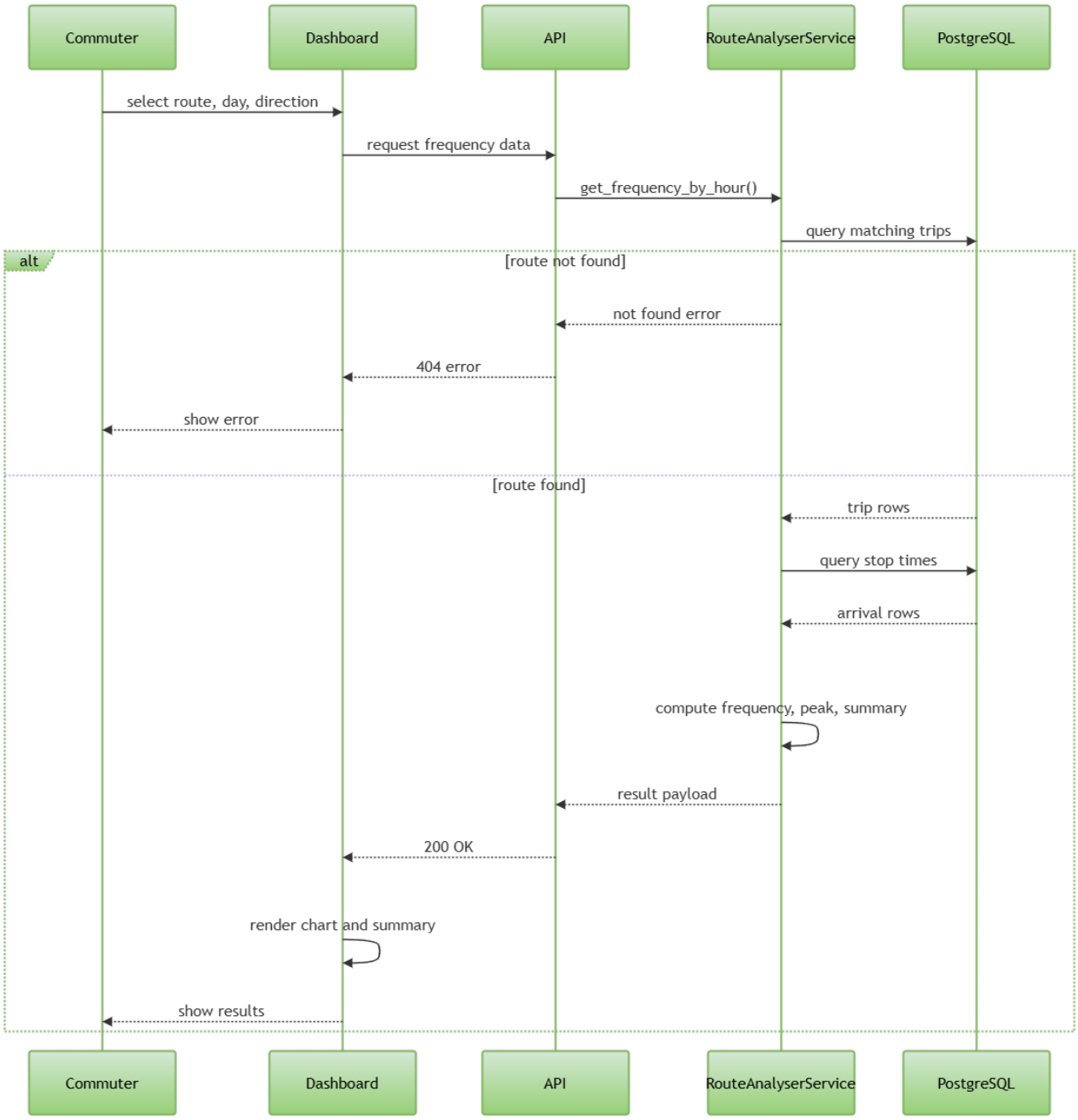


Figure 7: Route Analyser Sequence Diagram

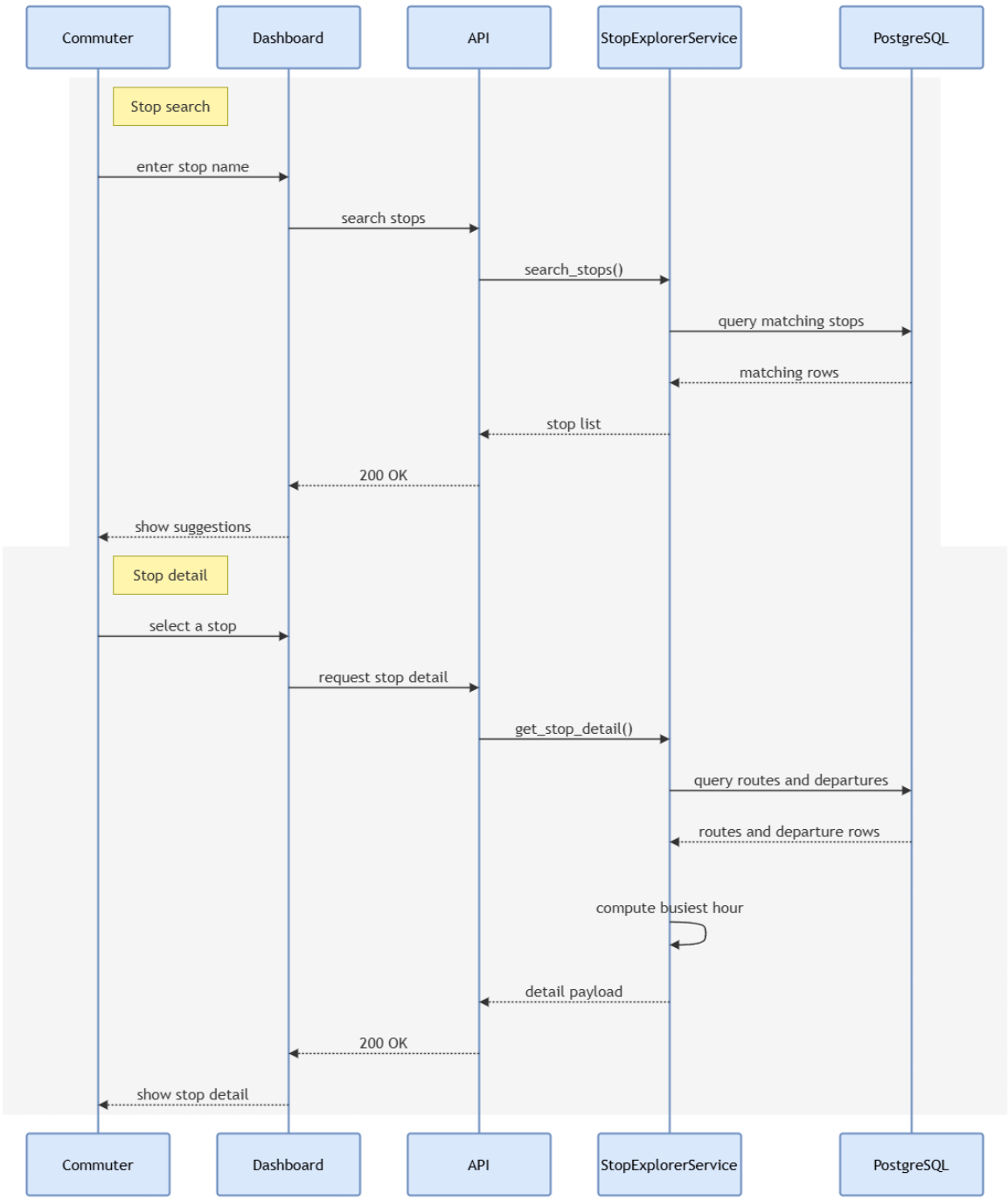


Figure 8: Stop Explorer Sequence Diagram

This Stop Explorer Sequence Diagram above covers two flows. In Stop Search, the commuter enters a stop name, which triggers search_stops() to query PostgreSQL for matching stops and return suggestions. In Stop Detail, selecting a specific stop triggers get_stop_detail(), which queries routes and departures for that stop, computes the busiest hour, and returns the full detail payload to the dashboard.

Wireframes are complete for all five dashboard pages, Network Overview, Route Analyser, Stop Explorer, Trip Planner Lite, and Network Patterns, in both Desktop and Mobile layouts. Desktop wireframes for each page are included below; the full wireframe files (Desktop and Mobile) are linked in the Appendix.

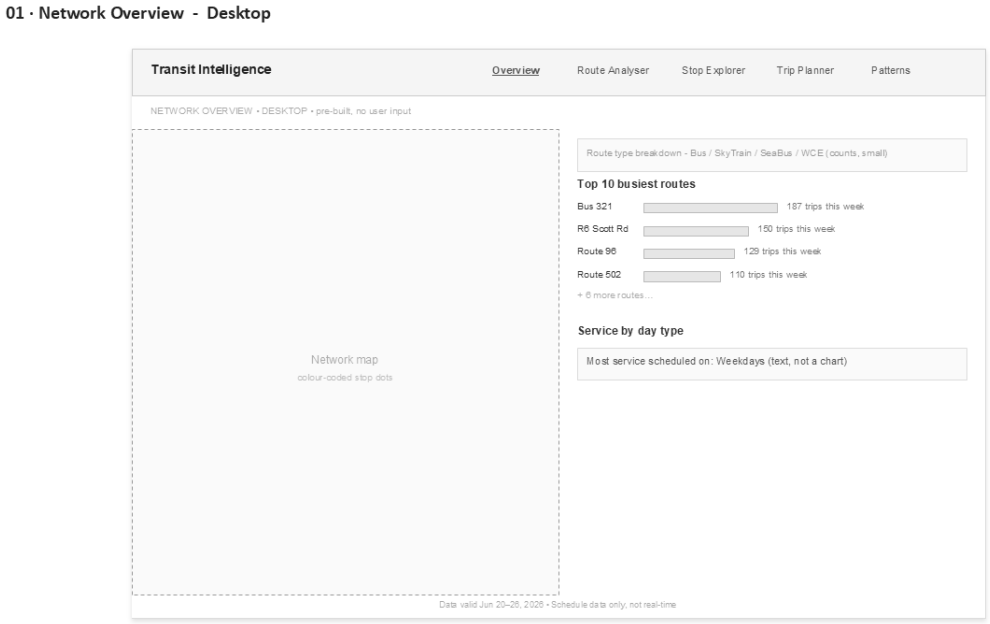


Figure 9: Desktop Wireframe- Network Overview

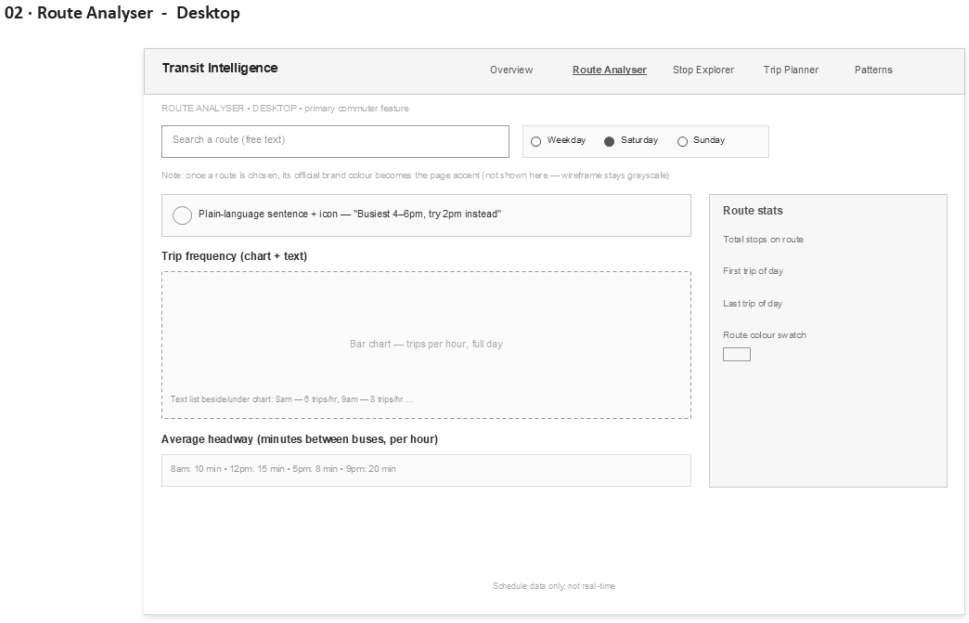


Figure 10: Desktop Wireframe- Route Analyser

03 · Stop Explorer - Desktop

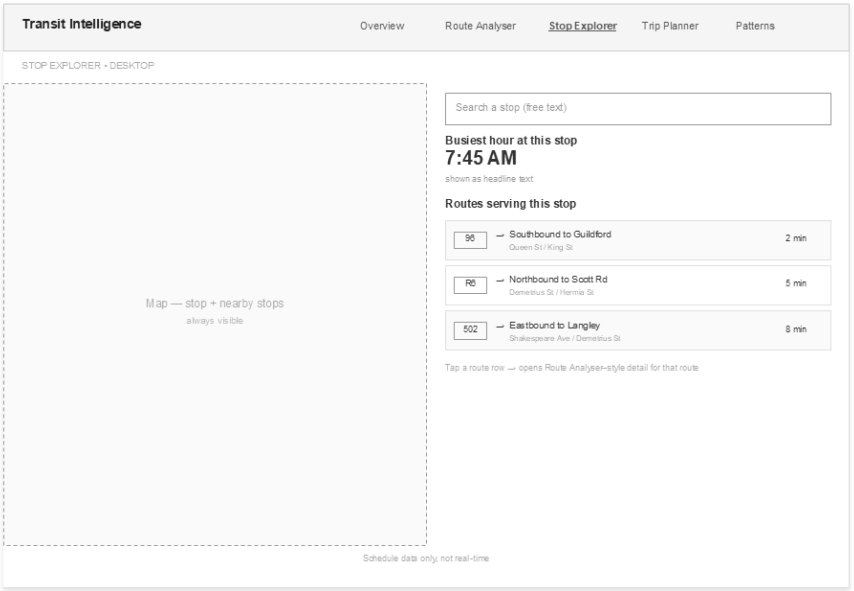


Figure 11: Desktop Wireframe- Stop Explorer

04 · Trip Planner Lite - Desktop

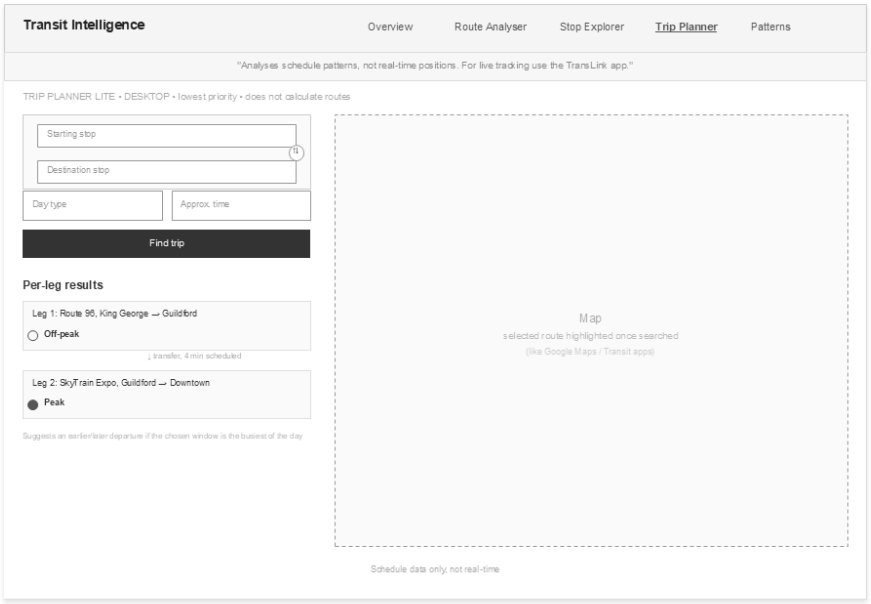


Figure 12: Desktop Wireframe: Trip Planner Lite

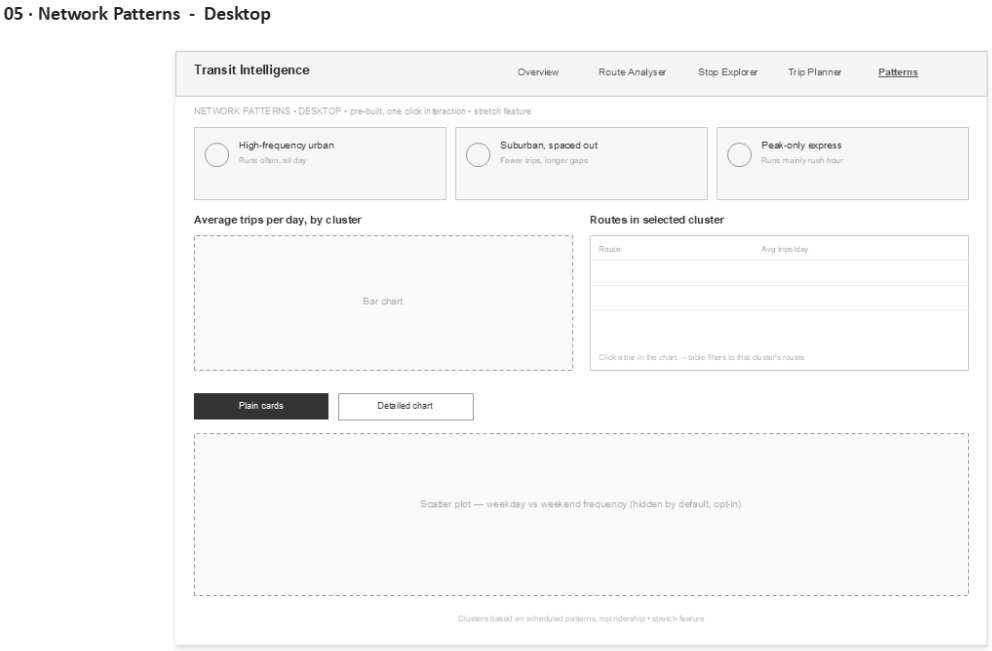


Figure 13: Desktop Wireframe- Network Patterns

The only outstanding design work is high-fidelity mockups (final colour, typography, and visual polish), which are scheduled for Week 10 per the course timeline and are not part of this report.

3. Development Planning

Implementation formally begins next semester in INFO 4290, so this section covers what has been planned and validated so far, not code written for the live system.

The build will follow the three-layer roadmap agreed since Progress Report 1: the Data layer first (automated GTFS download, parsing, and PostgreSQL load), then the Backend layer (Flask API, Route Search, Stop Explorer, Route Analyser endpoints), then the Frontend layer (dashboard shell, charts, plain-language summaries). Stretch features, K-Means clustering (optional, not a core requirement), accessibility filtering, and route comparison, are scoped last and will only be pursued if time allows after the core system is working. Tools are confirmed and unchanged since the proposal: Python, Pandas, PostgreSQL, Flask, Chart.js, and Scikit-Learn for the optional K-Means component. Design documentation has been version-controlled in a shared OneDrive folder throughout this semester; the GitHub repository for implementation is already created, with all submitted design documents pushed to it ahead of the INFO 4290 build phase.

Beyond the Week 1 GTFS validation script (which confirmed 242 routes and 63,250 trips load cleanly), the database schema itself has now been built and tested against real

TransLink data in a live PostgreSQL instance using a cloud-hosted PostgreSQL environment for validation purposes, strictly for development testing and not part of the final deployment design. the system design specifies PostgreSQL generally, not any particular hosting provider. This testing used the full weekly dataset rather than a small sample, so that any problems that only appear at scale would surface now, ahead of the build semester. What this testing found is detailed in Technical Challenges below.

For next semester, Inderpreet will lead backend development, including the GTFS import pipeline, PostgreSQL database, ETL/transform logic, Flask API, and optional K-Means clustering model. Drishti will lead frontend development, including implementation of the Chart.js dashboard, UI integration based on completed wireframes, user testing coordination, and documentation support. This continues the same division of labour that worked through the design phase, with Inderpreet's scope expanding to reflect the database validation work already completed.

4. Technical Challenges

Testing the database design against real TransLink data, rather than waiting until implementation, surfaced three genuine issues worth documenting.

The first was a mismatch between our original schema and TransLink's actual file structure: an early load attempt failed because TransLink's files include more columns than our simplified ERD assumed, and dates arrive in a non-standard format (e.g. 20260420). Rather than manually cleaning files by hand every week, which isn't sustainable, we redesigned the database into two layers, a staging layer that accepts TransLink's files exactly as published, and a curated layer populated by one reusable transform script. This means new weekly data now loads with zero manual prep.

The second was a real data integrity issue: TransLink's directions.txt file references 10 route IDs that don't exist in routes.txt, most likely leftover entries for discontinued or seasonal routes. The fix was to load only direction records with a valid matching route and log the rest, rather than let one bad reference fail the entire import.

The third was two design errors the ERD didn't catch until tested: stop_times needed to store arrival and departure times as text rather than a standard time value, since TransLink schedules some trips past midnight; and the shapes table needed a two-part primary key, since one shape has many GPS points. Both diagrams have been corrected.

Issue	Cause	Resolution
Schema didn't match raw TransLink files	Extra columns, non-standard date format	Split database into staging (raw) and curated (transformed) layers

directions.txt referenced missing routes	Orphaned/discontinued route IDs in TransLink's data	Only load directions with a valid route match; log the rest
Two ERD errors found under testing	Midnight-crossing trip times; shapes needing a composite key	Corrected both diagrams to match tested schema

5. Response to Peer Review #1

Team 8 (Jimoh Samira and Ify Ijeh) reviewed our Progress Report 1 and rated it highly, scoring 5/5 on Clarity of Scope, Actionable Next Steps, and Overall Organization. They gave two specific, actionable suggestions, both of which have since been addressed.

Their first suggestion was to include a system architecture diagram earlier in the project rather than waiting for a later deliverable. A simplified four-layer version was added to the Midterm Report, and it has since been expanded into the full, detailed architecture diagram included in this report as Figure 4.

Their second suggestion was to strengthen the risk section with user adoption and usability risks, not just technical ones. We added a new risk covering low user adoption due to unfamiliar terminology, mitigated through the plain-language summary feature and the Week 7+ user testing plan (8 participants, 6/8 pass threshold). These changes improved both system design completeness and usability clarity, particularly in how users interpret transit patterns.

6. Next Steps

- Finalize high-fidelity mock-ups for all five dashboard pages.
- Complete the Flask API endpoint specification and data dictionary
- Begin drafting the Final Report structure
- Prepare for Peer Review #2, including a defense of the staging/curated database redesign and other changes made during testing
- Continue database testing as needed to confirm the design is fully implementation-ready
- Confirm final task division with Drishti ahead of the INFO 4290 build phase
- Begin preparing final presentation materials (slides, demo walkthrough of wireframes and diagrams) ahead of Week 13

7. Appendix

1. Figure 1: Database ERD
2. Figure 2: Domain Class Diagram
3. Figure 3: Service Importer Class Diagram
4. Figure 4: System Architecture Diagram
5. Figure 5: ETL Pipeline Flowchart
6. Figure 6: Use Case Diagram
7. Figure 7 : Route Analyser Sequence Diagram
8. Figure 8: Stop Explorer Sequence Diagram
9. Full Desktop Wireframes file: Desktop Wireframes.pptx
10. Full Mobile Wireframes file: Mobile Wireframes.pptx
11. Database testing SQL scripts: translink_schema.sql, staging_schema.sql, transform_to_curated.sql